

중간고사 대체 과제 보고서



과 목 명	딥러닝/클라우드 2분반
교 수 님	오세종 교수님
제 출 일	2024.10.28
학 과	소프트웨어학과
학 번	32224684
이 름	최예림

목차

1. EDA.....	1
1) Train Dataset.....	1
데이터셋	1
시각화.....	1
이상치.....	5
정규성 검정.....	6
상관관계	7
VIF	8
2) Train 시계열 데이터	9
데이터 타입 변경	9
Target	10
정상성.....	12
3) Test 데이터.....	13
4) EDA 결과 정리	14
분석 결과 요약.....	14
모델의 유의점	14
2. Time Series Modeling.....	15
1) ARIMA.....	15
2) Prophet.....	17
3. ML Modeling.....	20
1) Base Model	20
Basic Data.....	21
Basic Time Data	22
One-Hot Encoding + PCA Data.....	23

데이터별 결과 비교	24
2) Feature Selection	25
Filter Method.....	25
Forward Selection	26
Backward Elimination	28
결과 비교	29
3) 중간 제출 결과 비교.....	30
4) 추가 테스트	31
Lasso	31
하이퍼파라미터 튜닝.....	32
Model Stacking	35
Voting.....	37
5) 추가 테스트 제출 결과 비교	39
4. DL Modeling.....	41
1) Sequential	41
모델	41
결과 비교	43
2) 제출 결과	44
5. 최종 결과	45
1) 제출 결과	45
2) 경진대회 참여 소감	46

1. EDA

모델링 전 데이터에 대한 이해를 위해 EDA 기법을 사용하여 전반적인 분석을 진행했다. train 데이터셋을 모델 학습 시 사용하기 때문에 train 데이터를 위주로 시각화 및 다양한 통계적 분석을 진행하였다. test 데이터는 시계열 데이터의 분석여부를 확인하기 위해 분포를 살펴보았다.

1) Train Dataset

데이터셋

```
import pandas as pd
train = pd.read_csv('./train.csv')
train.head()
```

	yymm	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V18	V19	V20	V21	V22	V23	V24	V25	V26	Target
0	0301 00:00	-5.327	12.250	-3.294	-7.855	-1.196	13.824	-10.249	-3.04	-5.17	...	0.103	1.001	-5.861	-27.695	-9.978	-2.689	-0.951	-3.873	0.471	44.521
1	0301 00:10	-5.267	12.916	-3.220	-7.788	-1.196	14.424	-10.249	-3.04	-4.97	...	0.073	0.935	-5.881	-37.695	-10.038	-2.652	-1.018	-3.503	0.361	35.027
2	0301 00:20	-5.127	13.583	-3.130	-7.658	-1.196	15.081	-10.359	-3.04	-4.83	...	0.013	0.905	-5.891	-37.695	-10.001	-2.652	-1.051	-3.436	0.361	13.920
3	0301 00:30	-5.060	14.250	-3.130	-7.532	-1.196	14.961	-10.359	-3.04	-4.83	...	-0.020	0.845	-5.911	-37.695	-10.028	-2.552	-1.111	-3.346	0.261	28.410
4	0301 00:40	-4.967	14.916	-3.094	-7.462	-1.196	15.454	-10.359	-3.04	-4.97	...	-0.087	0.811	-5.931	-37.695	-10.111	-2.619	-1.141	-3.346	0.261	1.647

5 rows × 28 columns

데이터셋을 불러와 데이터를 살펴보고 train.columns, train.info(), train.describe()를 사용하여 데이터셋의 피쳐 및 분포를 확인해보았다. train 데이터는 27개의 피쳐와 4464개의 행으로 되어 있다. 그 중 train 데이터셋에서 예측해야 하는 값은 Target 컬럼이다. Target 컬럼은 float64 형으로 회귀 모델을 사용해야 한다. 모델링에는 yymm과 V1 - V26 피쳐를 활용할 수 있다. yymm은 시계열 데이터로 mmdd hh:mm의 형식으로 작성되어 object형으로 저장되어 있다. 따라서 시계열 데이터로 분석하기 위해선 yymm 컬럼을 datetime과 같은 시계열 데이터로 변환할 필요가 있어 보였다.

시각화

Target 와 시계열 데이터가 아닌 V1 - V26 피쳐들을 시각화해보았다. 이들은 모두 수치형 데이터이기 때문에 각 데이터의 히스토그램, 박스그래프를 그려 분포를 확인하였다. Target 데이터의 경우 바이올린 플롯을 통해 분포를 더 살펴보고, 다른 피쳐들은 산점도 그래프를 통해 Target과의 상관관계를 확인하였다.

Target 데이터 시각화

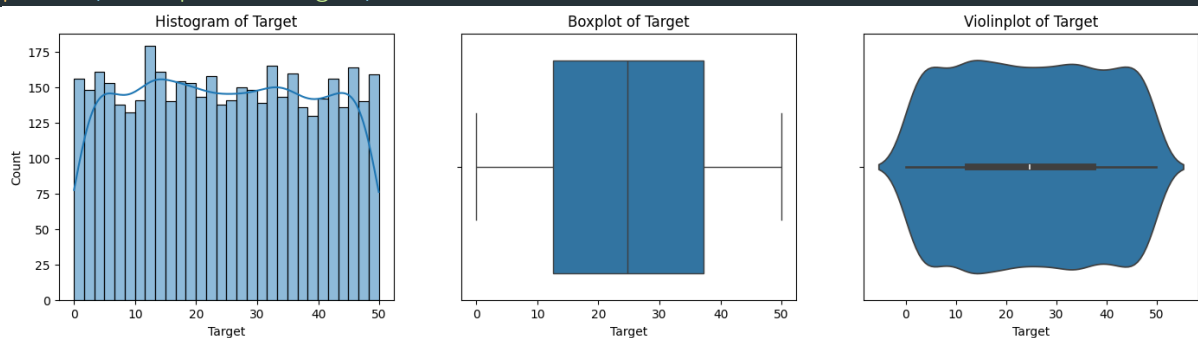
```
# Target 데이터 시각화
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(17, 4))

# 히스토그램
plt.subplot(1, 3, 1)
sns.histplot(train['Target'].dropna(), bins=30, kde=True)
plt.title('Histogram of Target')

# 박스플롯
plt.subplot(1, 3, 2)
sns.boxplot(x=train['Target'])
plt.title('Boxplot of Target')

# 바이올린 플롯
plt.subplot(1, 3, 3)
sns.violinplot(x=train['Target'])
plt.title('Violinplot of Target')
```



V1 - V26 시각화

```
import matplotlib.pyplot as plt
import seaborn as sns

# 시각화할 피쳐
columns = train.drop(['yymm', 'Target'], axis=1).columns

# 행과 열의 개수를 설정 (가로 3, 세로 26)
num_cols = 3
num_rows = 26

# 그래프 크기 설정
```

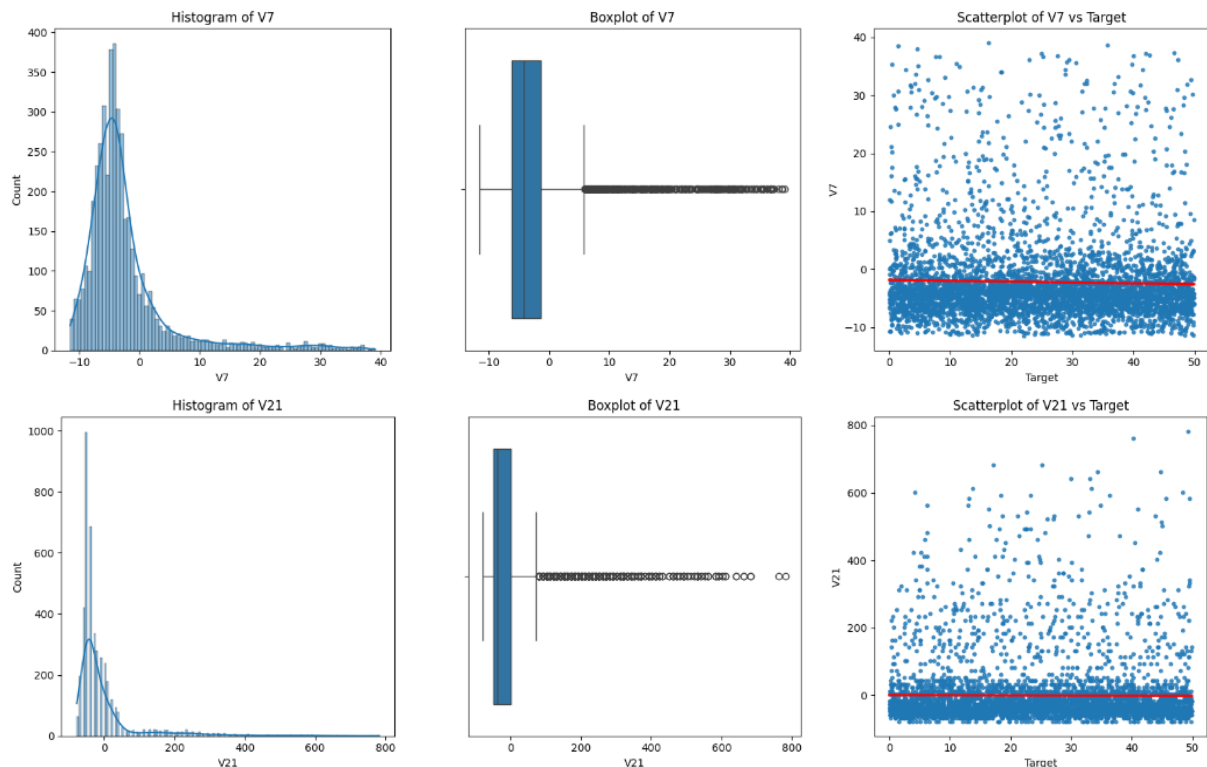
```
plt.figure(figsize=(16, num_rows * 5))

# 각 피처에 대해 히스토그램, 박스플롯, 산점도를 그리기
for i, col in enumerate(columns, 1):
    # 히스토그램
    plt.subplot(num_rows, num_cols, (i - 1) * 3 + 1)
    sns.histplot(train[col], kde=True)
    plt.title(f"Histogram of {col}")

    # 박스플롯
    plt.subplot(num_rows, num_cols, (i - 1) * 3 + 2)
    sns.boxplot(x=train[col])
    plt.title(f"Boxplot of {col}")

    # 산점도 (Target과) - 경향성을 나타내는 회귀선 추가
    plt.subplot(num_rows, num_cols, (i - 1) * 3 + 3)
    sns.regplot(x=train['Target'], y=train[col], scatter_kws={"s": 10}, line_kws={"color":
"red"})
    plt.title(f"Scatterplot of {col} vs Target")

# 그래프 출력
plt.tight_layout()
plt.show()
```



V1부터 V26까지 히스토그램, 박스플롯, 산점도 그래프를 그린 결과 피처들이 정규성을 보이지 않는다는 것을 알 수 있었다. 또한 V7, V21 등 일부 피처들이 이상치를 포함하고

있으며, 피쳐들이 Target과 큰 상관관계를 보이지 않는다는 것을 알 수 있다. 그래프에서 확인한 내용을 더 확실하게 확인하기 위해 이상치, 상관관계, 정규성을 확인해보았다.

이상치

```
from scipy.stats import zscore
import pandas as pd

# IQR를 사용한 이상치 함수
def check_outlier_iqr(df, columns):
    outliers = []
    for col in columns:
        q1 = df[col].quantile(0.25)
        q3 = df[col].quantile(0.75)
        iqr = q3 - q1
        lower_bound = q1 - (iqr * 1.5)
        upper_bound = q3 + (iqr * 1.5)
        outlier_cnt = len(df[(df[col] < lower_bound) | (df[col] > upper_bound)])
        if outlier_cnt > 0:
            outliers.append([col, outlier_cnt])
    outliers_df = pd.DataFrame(outliers, columns=['Feature', 'iqr_outliers'])
    outliers_df.set_index('Feature', inplace=True)
    return outliers_df

# zscore를 사용한 이상치 함수
def check_outlier_zscore(df, columns):
    outliers = []
    for col in columns:
        z_scores = zscore(df[col])
        outlier_cnt = len(df[(z_scores > 3) | (z_scores < -3)])
        if outlier_cnt > 0:
            outliers.append([col, outlier_cnt])
    outliers_df = pd.DataFrame(outliers, columns=['Feature', 'z_outliers'])
    outliers_df.set_index('Feature', inplace=True)
    return outliers_df

# 이상치 확인
columns = train.drop(['yymm', 'Target'], axis=1).columns
iqr = check_outlier_iqr(train, columns)
z_score = check_outlier_zscore(train, columns)
outliers = pd.concat([iqr, z_score], axis=1)
outliers.fillna(0, inplace=True)
outliers['z_outliers'] = outliers['z_outliers'].astype(int)

# 이상치 결과 출력
outliers
```


	iqr_outliers	z_outliers
Feature		
V1	1	9
V3	64	13
V4	3	0
V7	407	141
V8	28	25
V9	90	44
V11	1028	163
V12	11	17
V15	1	3
V16	23	13
V19	73	4
V21	460	124
V24	8	0

각 피쳐의 이상치를 구해본 결과 V7, V11, V21가 이상치를 많이 가지고 있음을 알 수 있다. iqr을 기준으로 했을 때 전체 데이터가 4464개이므로 V11은 약 23%, V7, V21은 약 10%가 이상치이기 때문에 이상치를 처리할 필요가 있을 것으로 판단했다.

정규성 검정

```
import pandas as pd
from scipy import stats

# 결과를 저장할 리스트 생성
results = []

# 정규성 검정 수행
for col in train.drop(['yymm'], axis=1).columns:
    # Shapiro-Wilk 테스트 수행
    w_stat, p_value = stats.shapiro(train[col])

    # p-value가 0.05보다 크면 정규성을 따른다고 간주
    is_normal = p_value > 0.05

    # 결과 리스트에 추가
    results.append([col, w_stat, p_value, is_normal])

# 결과 리스트를 데이터프레임으로 변환
normality_results = pd.DataFrame(results, columns=['Feature', 'W-statistic', 'p-value', 'Is Normal'])
normality_results.set_index('Feature', inplace=True)

# 결과 출력
```

normality_results								
	W-statistic	p-value	Is Normal					
Feature								
V1	0.976285	1.208986e-26	False		V13	0.980442	2.355369e-24	False
V2	0.945629	9.465682e-38	False		V14	0.992941	4.627120e-14	False
V3	0.989717	1.660614e-17	False		V15	0.962729	1.916527e-32	False
V4	0.994242	2.283935e-12	False		V16	0.987120	9.118638e-20	False
V5	0.935671	3.001522e-40	False		V17	0.945745	1.017050e-37	False
V6	0.947515	3.090351e-37	False		V18	0.992193	6.069940e-15	False
V7	0.710823	2.725897e-66	False		V19	0.990178	4.591776e-17	False
V8	0.923048	5.290412e-43	False		V20	0.989768	1.854919e-17	False
V9	0.977996	9.669296e-26	False		V21	0.581942	1.069645e-73	False
V10	0.962793	2.021191e-32	False		V22	0.993993	1.037977e-12	False
V11	0.548344	2.660585e-75	False		V23	0.991378	7.676820e-16	False
V12	0.966478	4.970867e-31	False		V24	0.992507	1.402390e-14	False
					V25	0.994019	1.127474e-12	False
					V26	0.989688	1.557934e-17	False
					Target	0.954519	3.447735e-35	False

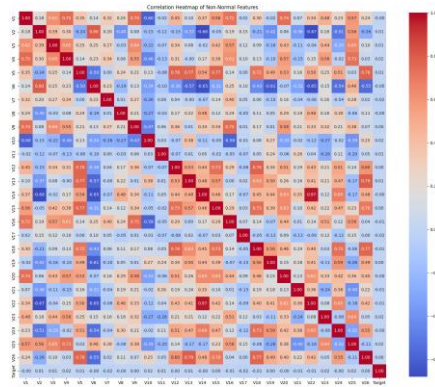
shapiro-wilk 검정을 사용해 정규성 검정을 진행한 결과 모든 피쳐들이 정규성을 만족하지 않는다. 따라서 correlation heatmap을 통해 상관관계를 확인할 때 'spearman' method를 사용했다.

상관관계

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# 상관관계 행렬 계산
correlation_matrix = train.drop('yymm', axis=1).corr(method='spearman')

# 히트맵 그리기
plt.figure(figsize=(20, 20))
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap='coolwarm', square=True,
            cbar_kws={"shrink": .8})
plt.title('Correlation Heatmap of Non-Normal Features')
plt.show()
```



correlation heatmap을보면 피쳐와 상관관계가 높은 피쳐가 없다. 하지만 피쳐들 간에는 0.7 이상의 강한 양의 상관관계와 -0.6 이상의 강한 음의 상관관계를 갖는 피쳐들이 존재한다. 특히, V21과 V14는 0.97로 매우 큰 상관관계를 갖는다. 상관관계가 높으면 다중공선성이 존재할 가능성이 높기 때문에 VIF를 확인해보았다.

VIF

```
import pandas as pd
from statsmodels.stats.outliers_influence import variance_inflation_factor

# 다중공선성
def is_mul(col):
    if col > 10:
        return True
    else:
        return False

# 스케일링 된 데이터
X = train.drop(['yymm', 'Target'], axis=1)

# VIF 계산
vif = pd.DataFrame()
vif['features'] = X.columns
vif['VIF'] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
vif.set_index('features', inplace=True)
vif['Multicollinearity'] = vif['VIF'].apply(is_mul)

# VIF 결과 출력
vif
```

	VIF	Multicollinearity			
features			V13	7.449847	False
			V14	368.431916	True
V1	16.565445	True	V15	8.811897	False
V2	139.656558	True	V16	10.406946	True
V3	11.903105	True	V17	1.148701	False
V4	8.608701	False	V18	14.124495	True
V5	11.572841	True	V19	4.016465	False
V6	8.138898	False	V20	205.521513	True
V7	1.355390	False	V21	1.267475	False
V8	1.758062	False	V22	28.589089	True
V9	12.799857	True	V23	7.535119	False
V10	2.486512	False	V24	27.388568	True
V11	1.344742	False	V25	9.188937	False
V12	8.685045	False	V26	5.524087	False

VIF가 10 이상이면 다중공선성이 존재한다고 판단한다. V2, V14, V20은 VIF가 100이 넘는 만큼 이 train 데이터셋에서는 다중공선성이 존재한다. 따라서 PCA 또는 피쳐 삭제를 통해 다중공선성을 해결하면 모델 성능을 높이는 데 도움이 될 수 있다.

2) Train 시계열 데이터

앞에서는 시계열 데이터인 yymm 컬럼을 제외하고 확인했기 때문에 이번에는 yymm을 시계열 데이터로 변환하여 살펴보았다.

데이터 타입 변경

```
import pandas as pd

# 데이터 로드
train = pd.read_csv('./train.csv')

# 연도를 임의로 설정하여 yymm 컬럼을 날짜 형식으로 변환
train['yymm'] = pd.to_datetime('2024' + train['yymm'], format='%Y%m%d %H:%M')

# day, hour, minute 컬럼 생성
train['day'] = train['yymm'].dt.day           # 일
train['hour'] = train['yymm'].dt.hour         # 시
train['minute'] = train['yymm'].dt.minute     # 분

# weekday 컬럼 생성 (요일 계산)
weekday = train['day'] % 7 # 0: 월요일, 1: 화요일, ..., 6: 일요일
weekday = weekday.map({0:'Mon', 1:'Tue', 2:'Wed', 3:'Thu', 4:'Fri', 5:'Sat', 6:'Sun'})
```

```
train['weekday'] = weekday
```

```
# 결과 출력
train.head(10)
```

	yymm	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V22	V23	V24	V25	V26	Target	day	hour	minute	weekday
0	2024-03-01 00:00:00	-5.327	12.250	-3.294	-7.855	-1.196	13.824	-10.249	-3.04	-5.170	...	-9.978	-2.689	-0.951	-3.873	0.471	44.521	1	0	0	Tue
1	2024-03-01 00:10:00	-5.267	12.916	-3.220	-7.788	-1.196	14.424	-10.249	-3.04	-4.970	...	-10.038	-2.652	-1.018	-3.503	0.361	35.027	1	0	10	Tue
2	2024-03-01 00:20:00	-5.127	13.583	-3.130	-7.658	-1.196	15.081	-10.359	-3.04	-4.830	...	-10.001	-2.652	-1.051	-3.436	0.361	13.920	1	0	20	Tue
3	2024-03-01 00:30:00	-5.060	14.250	-3.130	-7.532	-1.196	14.961	-10.359	-3.04	-4.830	...	-10.028	-2.552	-1.111	-3.346	0.261	28.410	1	0	30	Tue
4	2024-03-01 00:40:00	-4.967	14.916	-3.094	-7.462	-1.196	15.454	-10.359	-3.04	-4.970	...	-10.111	-2.619	-1.141	-3.346	0.261	1.647	1	0	40	Tue
5	2024-03-01 00:50:00	-4.967	15.583	-3.020	-7.388	-1.196	15.284	-10.419	-3.04	-4.860	...	-10.111	-2.689	-1.208	-3.346	0.171	6.360	1	0	50	Tue
6	2024-03-01 01:00:00	-4.827	16.250	-2.920	-7.288	-1.196	15.351	-10.449	-3.04	-4.933	...	-10.171	-2.762	-1.275	-3.346	0.171	34.535	1	1	0	Tue
7	2024-03-01 01:10:00	-4.797	16.250	-2.920	-7.222	-1.196	14.188	-10.516	-3.04	-4.860	...	-10.478	-2.689	-1.341	-3.206	0.071	21.335	1	1	10	Tue
8	2024-03-01 01:20:00	-4.737	16.250	-2.830	-7.188	-1.196	14.048	-10.659	-3.04	-4.933	...	-10.744	-2.689	-1.451	-3.073	0.004	34.687	1	1	20	Tue
9	2024-03-01 01:30:00	-4.900	16.250	-2.890	-7.188	-1.196	14.014	-10.659	-3.04	-4.430	...	-10.941	-2.792	-1.551	-2.706	0.038	34.136	1	1	30	Tue

10 rows × 32 columns

시계열 데이터를 분석하기 위해 yymm 컬럼을 datetime 형식으로 변경하였다. 이때 연도는 임시로 2024년으로 설정하였다. min(), max(), diff()를 사용하여 살펴본 결과 train 데이터는 3월 1일부터 3월 31일까지 10분 간격으로 수집된 데이터이다. 시간 순서대로 정렬되어 있다.

Target

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12, 10))

# day별
plt.subplot(4, 2, 1)
sns.lineplot(x='day', y='Target', data=train)
plt.title('Target by day')

plt.subplot(4, 2, 2)
sns.boxplot(x='day', y='Target', data=train)
plt.title('Target by day')

# hour별
plt.subplot(4, 2, 3)
sns.lineplot(x='hour', y='Target', data=train)
plt.title('Target by hour')

plt.subplot(4, 2, 4)
sns.boxplot(x='hour', y='Target', data=train)
plt.title('Target by hour')

# minute별
plt.subplot(4, 2, 5)
```

```

sns.lineplot(x='minute', y='Target', data=train)
plt.title('Target by minute')

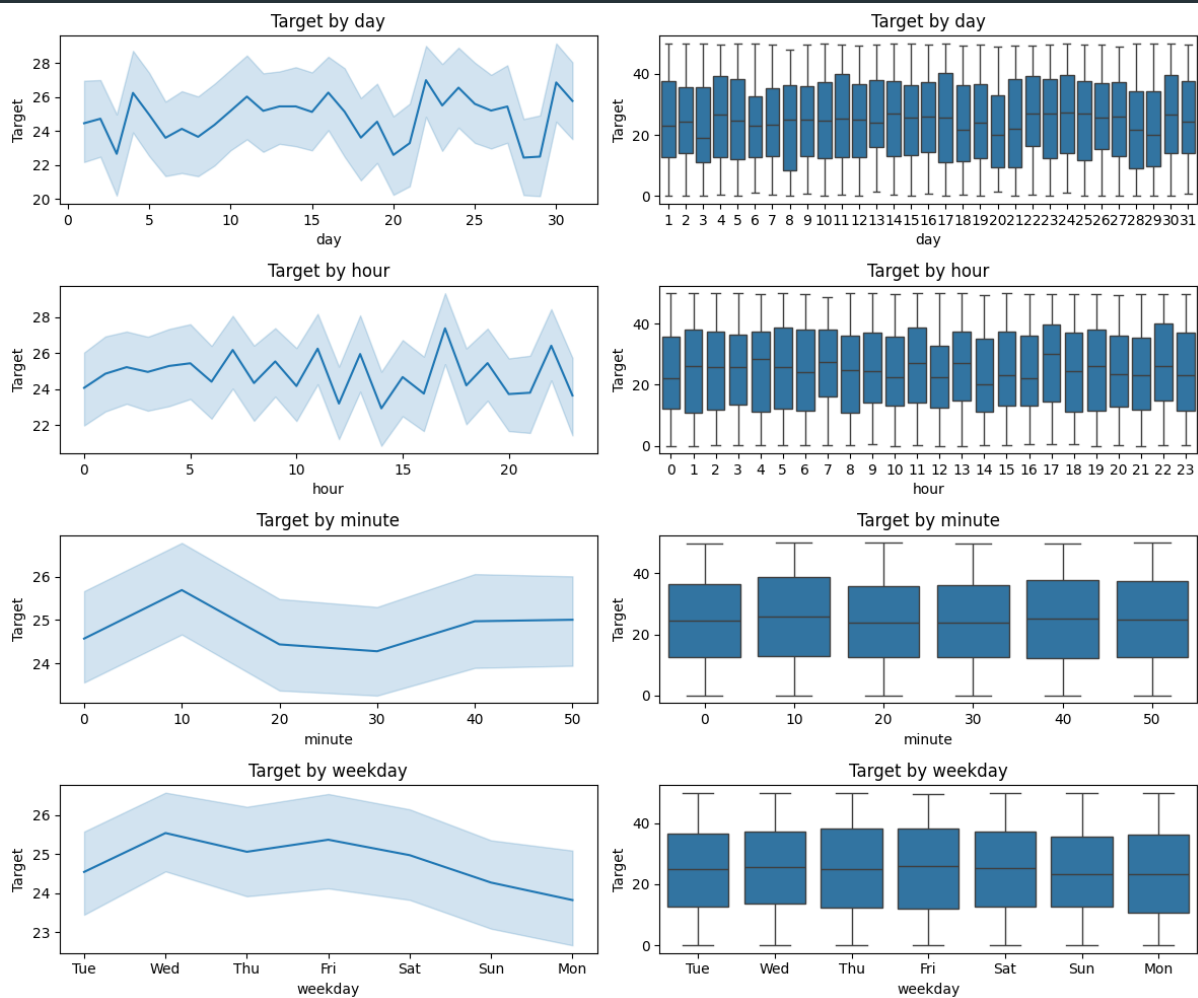
plt.subplot(4, 2, 6)
sns.boxplot(x='minute', y='Target', data=train)
plt.title('Target by minute')

# 요일별
plt.subplot(4, 2, 7)
sns.lineplot(x='weekday', y='Target', data=train)
plt.title('Target by weekday')

plt.subplot(4, 2, 8)
sns.boxplot(x='weekday', y='Target', data=train)
plt.title('Target by weekday')

# 그래프 출력
plt.tight_layout()
plt.show()

```



Target 데이터를 일, 시, 분, 요일을 기준으로 시각화하였다. 그래프를 확인해보면 시계열 특성에 따라 값이 크게 변하지 않지만, 특정 일 또는 시간에 평균 값이 높은 것은 확인할 수 있었다.

정상성

```
# 롤링 통계
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller

# 1. 시계열 플롯 (Target 컬럼)
plt.figure(figsize=(10, 6))
plt.plot(train['Target'], label='Target')
plt.title('Target Time Series')
plt.xlabel('Date')
plt.ylabel('Target Value')
plt.legend()
plt.show()

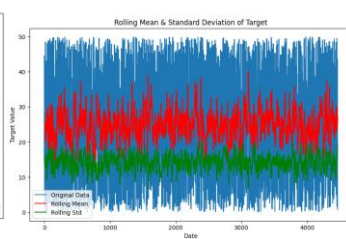
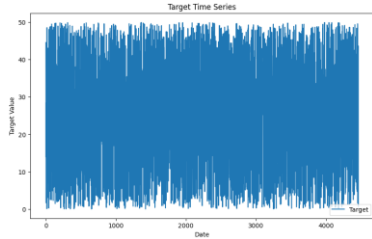
# 2. 롤링 평균 및 표준편차 시각화 (Window = 12)
rolling_mean = train['Target'].rolling(window=12).mean()
rolling_std = train['Target'].rolling(window=12).std()

plt.figure(figsize=(10, 6))
plt.plot(train['Target'], label='Original Data')
plt.plot(rolling_mean, label='Rolling Mean', color='red')
plt.plot(rolling_std, label='Rolling Std', color='green')
plt.title('Rolling Mean & Standard Deviation of Target')
plt.xlabel('Date')
plt.ylabel('Target Value')
plt.legend()
plt.show()

# 3. ADF(단위근) 검정 수행
result = adfuller(train['Target'])

# ADF 검정 결과 출력
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
print('Critical Values:')
for key, value in result[4].items():
    print('Wt%s: %.3f' % (key, value))
```

```
# 정상성 판정: p-value가 0.05보다 작으면 정상성을 가진다고 판단
if result[1] < 0.05:
    print("데이터는 정상성을 가지고 있습니다.")
else:
    print("데이터는 비정상성을 가지고 있습니다.")
```



```
ADF Statistic: -67.695949
p-value: 0.000000
Critical Values:
1%: -3.432
5%: -2.862
10%: -2.567
데이터는 정상성을 가지고 있습니다.
```

ADF와 KPSS로 정상성을 검정한 결과 train 데이터는 정상성을 가지고 있다. 따라서 ARIMA와 Prophet과 같은 시계열 예측 모델을 사용해 볼 수 있을 것으로 판단하였다.

3) Test 데이터

```
import pandas as pd

test = pd.read_csv('./test_set.csv')

# 연도를 임의로 설정하여 yymm 컬럼을 날짜 형식으로 변환
test['yymm'] = pd.to_datetime('2024' + test['yymm'], format='%Y%m%d %H:%M')

test.head()
```

	yymm	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V17	V18	V19	V20	V21	V22	V23	V24	V25	V26
0	2024-04-27 06:50:00	-4.094	15.750	0.110	-5.288	-0.196	1.548	-5.159	-0.706	-3.633	...	23.002	-1.053	-1.255	-2.244	-37.695	-4.644	-2.389	-2.841	-2.906	-0.239
1	2024-04-04 11:40:00	5.473	-5.417	3.110	3.172	1.114	-31.876	1.684	2.960	6.007	...	1.669	0.447	0.645	4.306	162.305	7.219	3.811	1.825	3.024	1.504
2	2024-04-24 19:30:00	-5.937	-11.750	-3.160	-5.874	0.904	-27.379	-9.449	0.960	-4.530	...	1.669	0.413	0.745	-2.911	2.305	-2.544	-8.022	-0.551	-6.936	2.294
3	2024-04-26 15:50:00	-4.327	-4.417	-2.954	-4.688	0.114	-14.582	-7.549	-0.373	-3.396	...	1.669	-0.897	-0.255	-2.727	-47.695	0.386	-6.389	-0.641	-5.036	-0.529
4	2024-05-13 13:40:00	4.633	-21.750	-1.420	5.487	5.014	-53.609	-1.889	0.627	6.530	...	-9.664	4.277	4.935	8.839	182.305	15.889	4.442	7.449	4.212	3.085

시계열 분석 모델을 사용할 수 있는지 판단하기 위해 test 데이터도 함께 살펴보았다. test 데이터는 4월 1일부터 5월 27일까지의 데이터를 포함하고 있다. test 데이터는 10분 간격으로 수집된 데이터이기 때문에 train과 같은 방식으로 수집된 데이터임을 알 수 있다. 하지만 train과 달리 정렬되지 않은 데이터이다. 또한 정렬을 해보았을 때 중간에 빈 시간대가 존재한다. 따라서 ARIMA와 같은 시계열 모델을 사용하게 된다면 test의 Target을 얻기 위해선 추가적인 과정이 필요할 것으로 예측된다.

4) EDA 결과 정리

분석 결과 요약

train 데이터는 시계열 회귀 데이터로 3월 1일부터 3월 31일까지 10분 간격으로 수집된 데이터이다. 정상성은 만족하지만 주기성은 크게 보이지 않는다. 이상치가 존재하는 피쳐들이 존재한다. 또한 모든 피쳐들은 정규성을 만족하지 않는다. 피쳐들과 target 간의 상관관계는 높지 않지만, 피쳐들 간의 상관성이 높아 다중공선성 문제 존재한다. test 데이터는 4월 1일부터 5월 27일까지의 데이터로 데이터가 정렬되지 않았고, 중간에 빈 시간대가 존재한다.

모델의 유의점

이상치의 비율이 큰 피쳐들이 있기 때문에 이상치 처리 방법을 시도해보아야 한다. 정규성을 만족하지 않기 때문에 StandardScaler()를 사용한 정규화를 진행할 수도 있고, 다중공선성 문제 해결을 위해 PCA 또는 피쳐 삭제를 진행해야한다. 시계열 회귀 문제이기 때문에 Regressor 모델들을 사용해야한다. 정상성을 만족하고 test가 train의 미래 데이터이기 때문에 ARIMA와 같은 시계열 예측 데이터를 사용할 수도 있다.

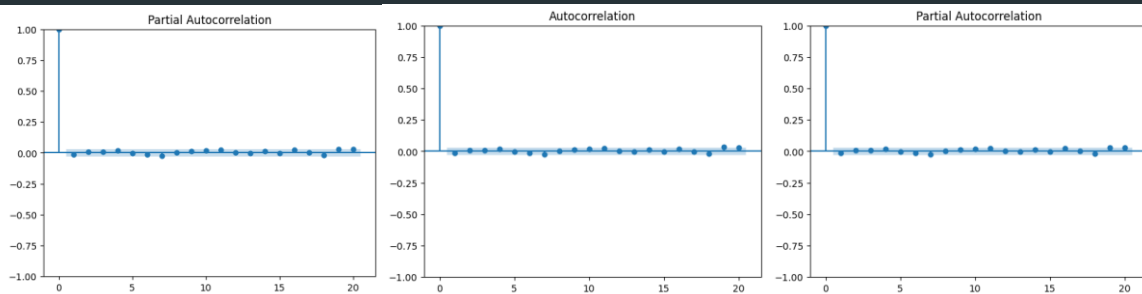
2. Time Series Modeling

Train 데이터는 정상성을 만족하기 때문에 시계열 데이터를 사용할 수 있다. 그래서 시계열 데이터를 테스트해보았다.

1) ARIMA

```
# 자기상관함수 도표를 통한 검증
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

# 자기상관함수 도표
plot_acf(train['Target'], lags=20)
plot_pacf(train['Target'], lags=20)
```



```
import pandas as pd
import matplotlib.pyplot as plt
from pmdarima import auto_arima

# 데이터 로드
train = pd.read_csv('./train.csv')

# yymm 컬럼을 날짜 형식으로 변환 (연도를 1900으로 설정)
train['yymm'] = pd.to_datetime('1900' + train['yymm'], format='%Y%m%d %H:%M')

# yymm을 인덱스로 설정
train = train.set_index('yymm')
train = train['Target']

# 주기(frequency) 설정
train.index.freq = '10min'

# ARIMA 모델 학습
model = auto_arima(train, seasonal=False, trace=True, stepwise=True)

# 4월 1일부터 5월 30일까지 예측
start_date = pd.Timestamp('1900-04-01 00:00')
```

```

end_date = pd.Timestamp('1900-05-30 23:50')
future_periods = (end_date - train.index[-1]) // pd.Timedelta(minutes=10) # 예측할
기간의 10분 단위 길이 계산

# 예측할 기간이 양수인지 확인
if future_periods > 0:
    # ARIMA 모델로 미래 예측
    forecast = model.predict(n_periods=future_periods)

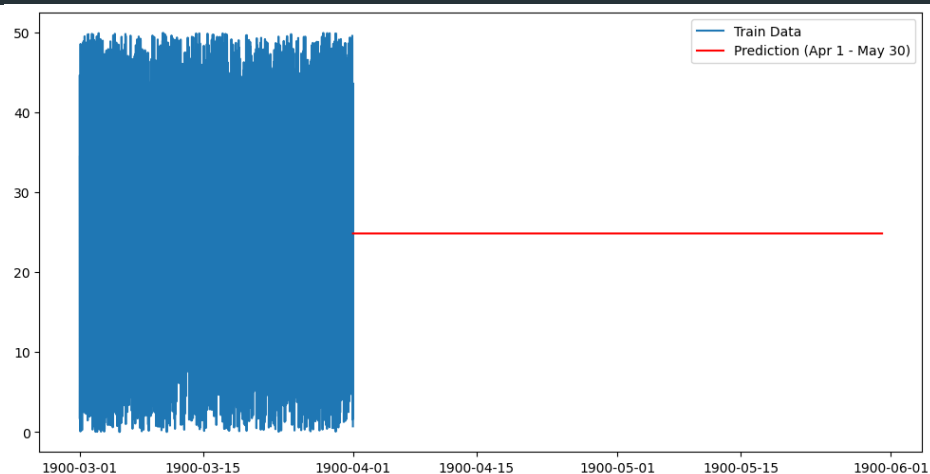
    # 예측 기간에 대한 인덱스 생성
    future_index = pd.date_range(start=train.index[-1] + pd.Timedelta(minutes=10),
periods=future_periods, freq='10min')
    forecast_series = pd.Series(forecast, index=future_index)

    # 예측값 중 4월 1일부터 5월 30일까지 선택
    forecast_filtered = forecast_series[(forecast_series.index >= start_date) &
(forecast_series.index <= end_date)]

    # 예측 결과 시각화
    plt.figure(figsize=(12, 6))
    plt.plot(train.index, train, label='Train Data')
    plt.plot(forecast_filtered.index, forecast_filtered, label='Prediction (Apr 1 - May 30)',
color='red')
    plt.legend()
    plt.show()

else:
    print("예측할 기간이 잘못되었습니다.")

```



ARIMA 함수들의 p , q , r 에 따라 가장 성능이 좋은 모델을 뽑고, test 데이터가 4월 1일부터 5월 27일까지 데이터르 포함하고 있기 때문에 4, 5월의 데이터를 예측해보았다. 그 결과 p , q , r 이 0, 0, 0인 모델이 가장 성능이 좋았다. 하지만 예측 결과를 보면 주어진 기간 동안

같은 값으로 예측하고 있음을 알 수 있다. 따라서 test 데이터에서 좋은 성능을 보이지 못할 것으로 평가하였다.

2) Prophet

```
import pandas as pd
import matplotlib.pyplot as plt
from prophet import Prophet
from prophet.plot import add_changepoints_to_plot
from sklearn.metrics import mean_absolute_error

# 데이터 로드
train = pd.read_csv('./train.csv')

# yymm 컬럼을 날짜 형식으로 변환 (연도를 1900으로 설정)
train['yyymm'] = pd.to_datetime('1900' + train['yyymm'], format='%Y%m%d %H:%M')

train = train.set_index('yyymm')
train = train['Target']

# 주기(frequency) 설정
train.index.freq = '10min'

# Prophet 모델 학습
model = Prophet()
model.fit(pd.DataFrame({'ds': train.index, 'y': train.values}))

# 4월 1일부터 5월 30일까지 예측
start_date = pd.Timestamp('1900-04-01 00:00')
end_date = pd.Timestamp('1900-05-30 23:50')
future_periods = (end_date - train.index[-1]) // pd.Timedelta(minutes=10) # 예측할
# 기간의 10분 단위 길이 계산

# 예측할 기간이 양수인지 확인
if future_periods > 0:
    # 미래 데이터 프레임 생성 및 예측
    future = model.make_future_dataframe(periods=future_periods, freq='10min')
    forecast = model.predict(future)

    # 예측 값 중 4월 1일부터 5월 30일까지 선택
    forecast_filtered = forecast[(forecast['ds'] >= start_date) & (forecast['ds'] <=
end_date)]

# 예측 결과 시각화
plt.figure(figsize=(12, 6))
plt.plot(train.index, train, label='Train Data')
```

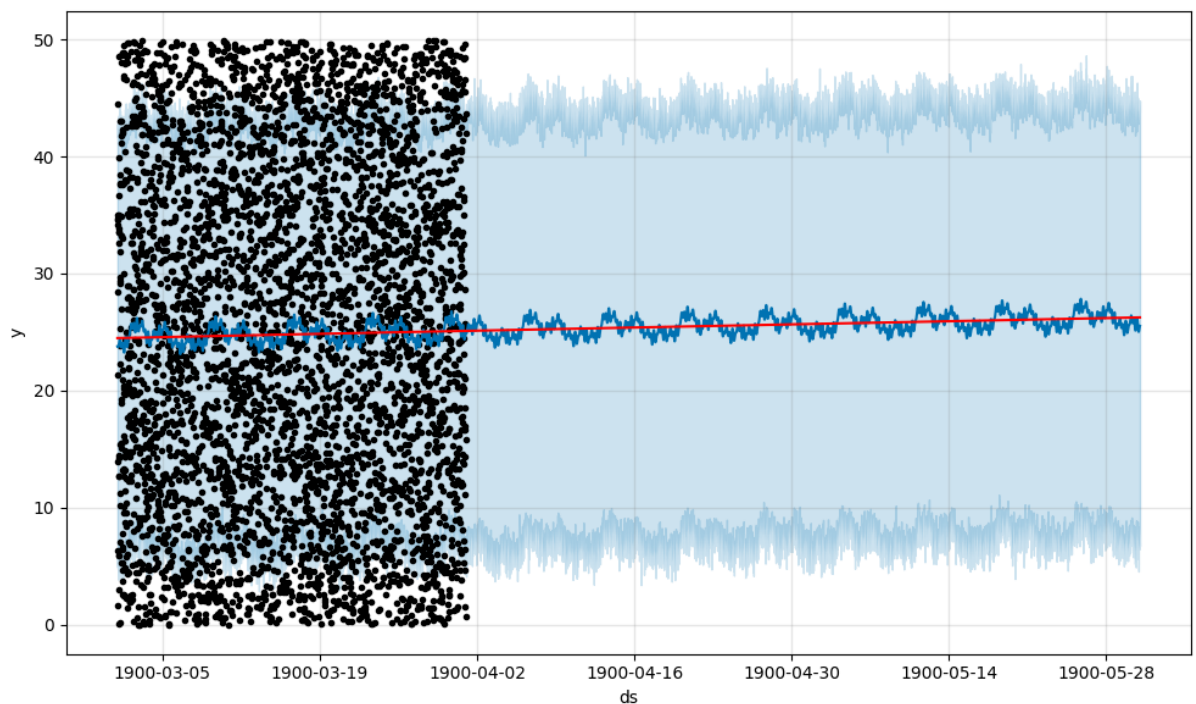
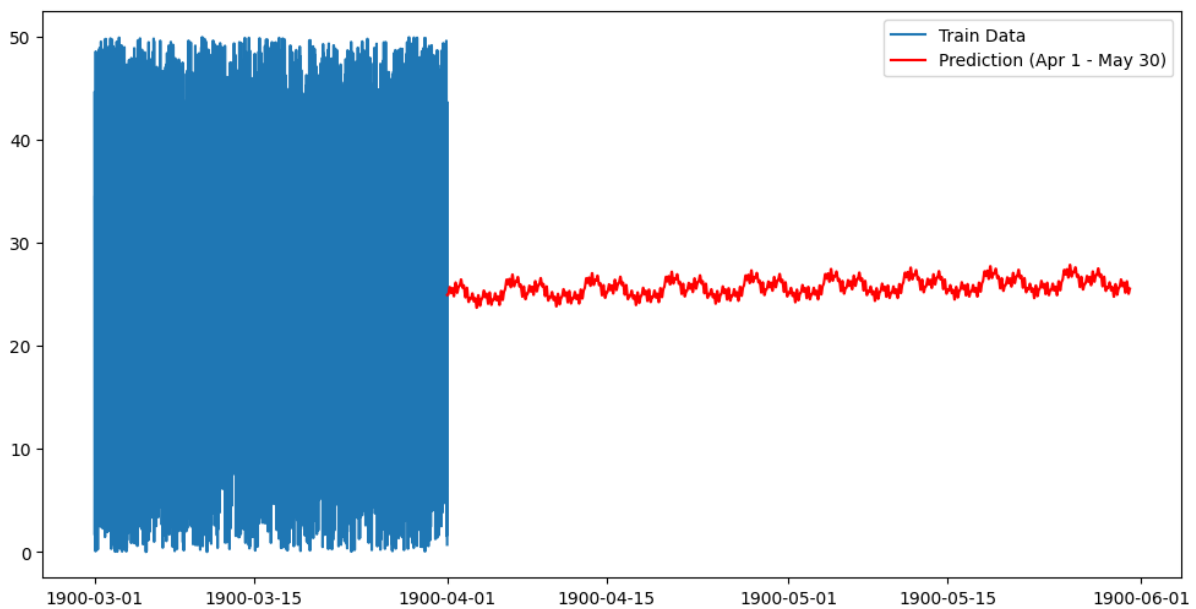
```

plt.plot(forecast_filtered['ds'], forecast_filtered['yhat'], label='Prediction (Apr 1 - May 30)', color='red')
plt.legend()
plt.show()

# Prophet 모델의 changepoint 시각화
fig = model.plot(forecast)
a = add_changepoints_to_plot(fig.gca(), model, forecast)

plt.show()
e/se:
print("예측할 기간이 잘못되었습니다. future_period")

```



ARIMA에서 유의미한 예측을 할 수 없었지만, Prophet 모델을 사용하여 다시 예측을 해보았다. 그 결과 ARIMA와는 달리 하나의 값으로 예측하지 않는다는 것을 알 수 있었다. 그리고 Prophet의 특성상 예측값이 주기성을 갖는 것을 확인할 수 있다.

```
# test DataFrame에 target 컬럼 추가 (빈 값으로 초기화)
test['target'] = None

# yymm과 ds를 기준으로 join하여 forecast_filtered의 yhat 값을 test의 target 컬럼에 저장
test = test.merge(forecast_filtered[['ds', 'yhat']], left_on='yyymm', right_on='ds', how='left')

# yhat 값을 target 컬럼에 복사
test['target'] = test['yhat']

# 필요 없는 ds, yhat 컬럼 제거
test = test.drop(columns=['ds', 'yhat'])

test.head()
```

Prophet은 ARIMA보다 성능이 좋아 보였기 때문에 test 데이터의 날짜와 비교하여 값을 제출해보았다. 그 결과 12.60으로 괜찮은 성적을 보였다.

3. ML Modeling

Target은 수치형 데이터로 회귀 문제이다. 따라서 머신러닝 모델 중 회귀 모델인 Linear Regression, Ridge, Lasso, ElasticNet, SVR, Random Forest, Gradient Boosting, XGBoost, LightGBM, Decision Tree를 테스트해보았다. 각 모델의 성능은 5-fold 방식으로 mae를 평가 지표로 사용하여 평가하였다.

1) Base Model

```
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from xgboost import XGBRegressor
from lightgbm import LGBMRegressor
from sklearn.svm import SVR
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import make_scorer, mean_absolute_error

# 데이터 분할
X = train.drop('Target', axis=1)    # Target을 제외한 모든 컬럼을 X로 지정
y = train['Target']                # Target 컬럼을 y로 지정

# train, test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# 모델 정의
models = {
    'Linear Regression': LinearRegression(),
    'Ridge': Ridge(random_state=42),
    'Lasso': Lasso(random_state=42),
    'ElasticNet': ElasticNet(random_state=42),
    'SVR': SVR(),
    'Gradient Boosting': GradientBoostingRegressor(random_state=42),
    'Random Forest': RandomForestRegressor(random_state=42),
    'XGBoost': XGBRegressor(random_state=42),
    'LightGBM': LGBMRegressor(random_state=42, verbose=-1),
    'Decision Tree': DecisionTreeRegressor(random_state=42)
}

# MAE를 평가 기준으로 사용하기 위해 scorer 정의
mae_scorer = make_scorer(mean_absolute_error)

# 각 모델에 대해 학습 및 5-fold 교차검증 수행
```

```
for model_name, model in models.items():
    scores = cross_val_score(model, X_train, y_train, cv=5, scoring=mae_scorer)
    print(f'{model_name}: {scores.mean()}')
```

```
Linear Regression: 12.626007154563748
Ridge: 12.625872446923035
Lasso: 12.580224346550702
ElasticNet: 12.582955143638385
SVR: 12.564188226545891
Gradient Boosting: 12.63337062841707
Random Forest: 13.353987621984615
XGBoost: 13.89784202256629
LightGBM: 13.157735041902736
Decision Tree: 16.959262534358977
```

위의 코드를 사용하여 각 모델의 MAE 값을 출력하여 모델들의 성능을 비교하였다.
시계열 데이터인 yymm 컬럼을 처리하는 방법에 따라 나누어 평가해보았다.

Basic Data

```
import pandas as pd
```

```
# 데이터 로드
```

```
train = pd.read_csv('./train.csv')
```

```
# yymm 컬럼 삭제
```

```
train.drop('yymm', axis=1, inplace=True)
```

```
# 결과 출력
```

```
train.head(10)
```

	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	V11	V12	V13	V14	V15	V16	V17	V18	V19	V20	V21	V22	V23	V24	V25	V26	Target
0	-5.327	12.250	-3.294	-7.855	-1.196	13.824	-10.249	-3.04	-5.170	8.077	16.198	-1.101	-0.067	-8.412	-0.592	-4.153	23.669	0.103	1.001	-5.861	-27.695	-9.978	-2.689	-0.951	-3.873	0.471	44.521
1	-5.267	12.916	-3.220	-7.788	-1.196	14.424	-10.249	-3.04	-4.970	8.027	16.198	-1.168	-0.067	-8.532	-0.592	-4.079	21.669	0.073	0.935	-5.881	-37.695	-10.038	-2.652	-1.018	-3.503	0.361	35.027
2	-5.127	13.583	-3.130	-7.658	-1.196	15.081	-10.359	-3.04	-4.830	7.977	16.198	-1.168	-0.067	-8.642	-0.665	-3.953	19.669	0.013	0.905	-5.891	-37.695	-10.001	-2.652	-1.051	-3.436	0.361	13.920
3	-5.060	14.250	-3.130	-7.532	-1.196	14.961	-10.359	-3.04	-4.830	7.927	26.198	-1.168	-0.067	-8.762	-0.592	-3.953	17.669	-0.020	0.845	-5.911	-37.695	-10.028	-2.552	-1.111	-3.346	0.261	28.410
4	-4.967	14.916	-3.094	-7.462	-1.196	15.454	-10.359	-3.04	-4.970	7.877	16.198	-1.168	-0.067	-8.882	-0.629	-3.916	15.669	-0.087	0.811	-5.931	-37.695	-10.111	-2.619	-1.141	-3.346	0.261	1.647
5	-4.967	15.583	-3.020	-7.388	-1.196	15.284	-10.419	-3.04	-4.860	7.827	16.198	-1.168	-0.134	-8.992	-0.702	-3.916	13.669	-0.087	0.745	-5.941	-37.695	-10.111	-2.689	-1.208	-3.346	0.171	6.360
6	-4.827	16.250	-2.920	-7.288	-1.196	15.351	-10.449	-3.04	-4.933	7.777	16.198	-1.268	-0.167	-9.112	-0.702	-3.953	11.669	-0.153	0.695	-5.961	-47.695	-10.171	-2.762	-1.275	-3.346	0.171	34.535
7	-4.797	16.250	-2.920	-7.222	-1.196	14.188	-10.516	-3.04	-4.860	7.727	26.198	-1.268	-0.167	-9.242	-0.769	-3.983	9.169	-0.187	0.645	-6.111	-37.695	-10.478	-2.689	-1.341	-3.206	0.071	21.335
8	-4.737	16.250	-2.830	-7.188	-1.196	14.048	-10.659	-3.04	-4.933	7.677	16.198	-1.268	-0.167	-9.382	-0.802	-4.043	6.669	-0.260	0.645	-6.261	-37.695	-10.744	-2.689	-1.451	-3.073	0.004	34.687
9	-4.900	16.250	-2.890	-7.188	-1.196	14.014	-10.659	-3.04	-4.430	7.627	6.198	-1.268	-0.234	-9.512	-0.802	-4.013	4.169	-0.297	0.535	-6.411	-37.695	-10.941	-2.792	-1.551	-2.706	0.038	34.136

Basic Data는 시계열 데이터인 yymm 컬럼을 제외한 데이터셋이다.

Model	MAE	Model	MAE
Linear Regression	12.6260	Gradient Boosting	12.6333
Ridge	12.6258	RandomForest	13.3539
Lasso	12.5802	XGBoost	13.8978
ElasticNet	12.5829	LightGBM	13.1577
SVR	12.5641	DecisionTree	16.9592

Basic Data를 Basic Model 코드로 학습한 결과는 위의 표와 같다.

Basic Time Data

```
import pandas as pd

# 데이터 로드
train = pd.read_csv('./train.csv')

# yymm 컬럼을 날짜 형식으로 변환 (연도는 임의로 설정)
train['yymm'] = pd.to_datetime('2024' + train['yymm'], format='%Y%m%d %H:%M')

# day, hour, minute 컬럼 생성
train['day'] = train['yymm'].dt.day           # 일
train['hour'] = train['yymm'].dt.hour         # 시
train['minute'] = train['yymm'].dt.minute     # 분

# weekday 컬럼 생성
train['weekday'] = train['day'] % 7           # 요일 (0: 월요일, 1: 화요일, ..., 6: 일요일)

# yymm 컬럼 삭제
train.drop('yymm', axis=1, inplace=True)

# 결과 출력
train.head(10)
```

	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	...	V22	V23	V24	V25	V26	Target	day	hour	minute	weekday
0	-5.327	12.250	-3.294	-7.855	-1.196	13.824	-10.249	-3.04	-5.170	8.077	...	-9.978	-2.689	-0.951	-3.873	0.471	44.521	1	0	0	1
1	-5.267	12.916	-3.220	-7.788	-1.196	14.424	-10.249	-3.04	-4.970	8.027	...	-10.038	-2.652	-1.018	-3.503	0.361	35.027	1	0	10	1
2	-5.127	13.583	-3.130	-7.658	-1.196	15.081	-10.359	-3.04	-4.830	7.977	...	-10.001	-2.652	-1.051	-3.436	0.361	13.920	1	0	20	1
3	-5.060	14.250	-3.130	-7.532	-1.196	14.961	-10.359	-3.04	-4.830	7.927	...	-10.028	-2.552	-1.111	-3.346	0.261	28.410	1	0	30	1
4	-4.967	14.916	-3.094	-7.462	-1.196	15.454	-10.359	-3.04	-4.970	7.877	...	-10.111	-2.619	-1.141	-3.346	0.261	1.647	1	0	40	1
5	-4.967	15.583	-3.020	-7.388	-1.196	15.284	-10.419	-3.04	-4.860	7.827	...	-10.111	-2.689	-1.208	-3.346	0.171	6.360	1	0	50	1
6	-4.827	16.250	-2.920	-7.288	-1.196	15.351	-10.449	-3.04	-4.933	7.777	...	-10.171	-2.762	-1.275	-3.346	0.171	34.535	1	1	0	1
7	-4.797	16.250	-2.920	-7.222	-1.196	14.188	-10.516	-3.04	-4.860	7.727	...	-10.478	-2.689	-1.341	-3.206	0.071	21.335	1	1	10	1
8	-4.737	16.250	-2.830	-7.188	-1.196	14.048	-10.659	-3.04	-4.933	7.677	...	-10.744	-2.689	-1.451	-3.073	0.004	34.687	1	1	20	1
9	-4.900	16.250	-2.890	-7.188	-1.196	14.014	-10.659	-3.04	-4.430	7.627	...	-10.941	-2.792	-1.551	-2.706	0.038	34.136	1	1	30	1

10 rows x 31 columns

Basic Time Data는 시계열 데이터 피처를 포함한 데이터 셋으로 day, hour, minute, weekday 피처를 추가했다.

Model	MAE	Model	MAE
Linear Regression	12.6371	Gradient Boosting	12.6414
Ridge	12.6370	RandomForest	13.2603
Lasso	12.5856	XGBoost	13.9893
ElasticNet	12.5938	LightGBM	13.2710
SVR	12.5799	DecisionTree	17.0525

Basic Time Data를 Basic Model 코드로 학습한 결과는 위의 표와 같다.

One-Hot Encoding + PCA Data

```
import pandas as pd
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# 데이터 로드
train = pd.read_csv('./train.csv')

# yymm 컬럼을 날짜 형식으로 변환 (연도는 임의로 설정)
train['yymm'] = pd.to_datetime('2024' + train['yymm'], format='%Y%m%d %H:%M')

# day, hour, minute, weekday 컬럼 생성
day = train['yymm'].dt.day
day_dummies = pd.get_dummies(day, prefix='day')

hour = train['yymm'].dt.hour
hour_dummies = pd.get_dummies(hour, prefix='hour')

minute = train['yymm'].dt.minute
minute_dummies = pd.get_dummies(minute, prefix='minute')

# weekday 컬럼 생성
weekday = day % 7
weekday = weekday.map({0:'Mon', 1:'Tue', 2:'Wed', 3:'Thu', 4:'Fri', 5:'Sat', 6:'Sun'})
weekday_dummies = pd.get_dummies(weekday)

# PCA 피쳐 생성
features = train.loc[:, 'V1':'V26'] # V1 ~ V26 컬럼 선택

scaler = StandardScaler()
features = scaler.fit_transform(features) # 피쳐 표준화

pca = PCA(n_components=3)
pca_features = pca.fit_transform(features) # PCA 피쳐 생성

# 생성된 주성분을 DataFrame으로 변환
pca_columns = ['PCA1', 'PCA2', 'PCA3']
pca_df = pd.DataFrame(pca_features, columns=pca_columns)

# 원본 데이터와 PCA 피쳐 결합
train = pd.concat([train, day_dummies, hour_dummies, minute_dummies,
weekday_dummies, pca_df], axis=1)

# yymm 컬럼 삭제
```

```
train.drop('yymm', axis=1, inplace=True)
```

```
# 결과 출력
train.head(10)
```

	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	...	Fri	Mon	Sat	Sun	Thu	Tue	Wed	PCA1	PCA2	PCA3
0	-5.327	12.250	-3.294	-7.855	-1.196	13.824	-10.249	-3.04	-5.170	8.077	...	False	False	False	False	False	True	False	-2.616525	-2.341176	-3.016174
1	-5.267	12.916	-3.220	-7.788	-1.196	14.424	-10.249	-3.04	-4.970	8.027	...	False	False	False	False	False	True	False	-2.691167	-2.187795	-3.000616
2	-5.127	13.583	-3.130	-7.658	-1.196	15.081	-10.359	-3.04	-4.830	7.977	...	False	False	False	False	False	True	False	-2.724164	-2.076864	-2.954282
3	-5.060	14.250	-3.130	-7.532	-1.196	14.961	-10.359	-3.04	-4.830	7.927	...	False	False	False	False	False	True	False	-2.744811	-2.076954	-2.864929
4	-4.967	14.916	-3.094	-7.462	-1.196	15.454	-10.359	-3.04	-4.970	7.877	...	False	False	False	False	False	True	False	-2.827777	-1.960836	-2.942832
5	-4.967	15.583	-3.020	-7.388	-1.196	15.284	-10.419	-3.04	-4.860	7.827	...	False	False	False	False	False	True	False	-2.914471	-1.906830	-2.889545
6	-4.827	16.250	-2.920	-7.288	-1.196	15.351	-10.449	-3.04	-4.933	7.777	...	False	False	False	False	False	True	False	-3.008483	-1.854571	-2.893859
7	-4.797	16.250	-2.920	-7.222	-1.196	14.188	-10.516	-3.04	-4.860	7.727	...	False	False	False	False	False	True	False	-3.088857	-1.917965	-2.752764
8	-4.737	16.250	-2.830	-7.188	-1.196	14.048	-10.659	-3.04	-4.933	7.677	...	False	False	False	False	False	True	False	-3.212438	-1.820224	-2.834904
9	-4.900	16.250	-2.890	-7.188	-1.196	14.014	-10.659	-3.04	-4.430	7.627	...	False	False	False	False	False	True	False	-3.306377	-1.653918	-2.872736

10 rows × 98 columns

One-hot Encoding + PCA 데이터는 시계열 데이터를 get_dummies()를 사용하여

Model	MAE	Model	MAE
Linear Regression	12.7023	Gradient Boosting	12.6956
Ridge	12.6981	RandomForest	13.2089
Lasso	12.5802	XGBoost	14.0245
ElasticNet	12.5829	LightGBM	13.2544
SVR	12.5643	DecisionTree	16.9341

One-hot Encoding + PCA 데이터를 Basic Model 코드로 학습한 결과는 위의 표와 같다.

데이터별 결과 비교

Model	Basic	Model	Time	Model	Onehot
Linear Regression	12.6260	Linear Regression	12.6371	Linear Regression	12.7023
Ridge	12.6258	Ridge	12.6370	Ridge	12.6981
Lasso	12.5802	Lasso	12.5856	Lasso	12.5802
ElasticNet	12.5829	ElasticNet	12.5938	ElasticNet	12.5829
SVR	12.5641	SVR	12.5799	SVR	12.5643
Gradient Boosting	12.6333	Gradient Boosting	12.6414	Gradient Boosting	12.6956
RandomForest	13.3539	RandomForest	13.2603	RandomForest	13.2089
XGBoost	13.8978	XGBoost	13.9893	XGBoost	14.0245
LightGBM	13.1577	LightGBM	13.2710	LightGBM	13.2544

DecisionTree	16.9592	DecisionTree	17.0525	DecisionTree	16.9341
--------------	---------	--------------	---------	--------------	---------

모든 데이터에서 Random Forest, XGBoost, LightGBM, Decision Tree는 MAE 점수가 12점보다 크기 때문에 성능이 많이 떨어진다. 평균적인 성능을 보면 Basic Data가 가장 성능이 좋고, 다음으로 One-hot, Time의 순서로 성능이 좋다.

2) Feature Selection

5-fold를 기준으로 성능을 비교한 결과 피처의 개수가 적을수록 성능이 좋았다. 그래서 앞에서 성능이 좋았던 Linear Regression, Ridge, Lasso, ElasticNet, SVR, GradientBoosting을 기준으로 피처를 선정하여 성능을 높여보았다.

Filter Method

```
import numpy as np
import pandas as pd
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.svm import SVR
from sklearn.feature_selection import SelectKBest, f_regression
from sklearn.metrics import make_scorer, mean_absolute_error

# 데이터 분할
X = train.drop('Target', axis=1) # Target 컬럼 제외
y = train['Target'] # Target 컬럼

# 모델 정의
models = {
    'Linear Regression': LinearRegression(),
    'Ridge': Ridge(random_state=42),
    'Lasso': Lasso(random_state=42),
    'ElasticNet': ElasticNet(random_state=42),
    'SVR': SVR(),
    'Gradient Boosting': GradientBoostingRegressor(random_state=42)
}

# MAE를 평가 기준으로 사용하기 위해 scorer 정의
mae_scorer = make_scorer(mean_absolute_error)

# SelectKBest로 K-최고 특성 선택
test = SelectKBest(score_func=f_regression, k=X.shape[1])
```

```

fit = test.fit(X, y)

# 선택된 특성들의 인덱스를 내림차순으로 정렬
sorted_columns = np.argsort(fit.scores_)[::-1]

# 각 모델에 대해 최적의 특성 선택
for model_name, model in models.items():

    # 최적의 특성을 찾기 위한 변수 초기화
    best_score = float('inf')
    best_features = []

    # 최적의 특성 선택
    for i in range(1, X.shape[1] + 1):
        # 선택된 feature들의 인덱스
        fs = sorted_columns[:i]

        # 선택된 feature만 선택 (Pandas DataFrame에서 iloc 사용)
        X_selected = X.iloc[:, fs]

        # 선택된 feature들의 이름
        selected_feature_names = X.columns[fs].tolist()

        # 교차 검증
        mae = cross_val_score(model, X_selected, y, cv=5, scoring=mae_scorer).mean()

        # 가장 성능이 좋은 MAE 및 feature를 저장
        if mae < best_score:
            best_score = mae
            best_features = selected_feature_names

    # 결과 출력
    print(f'{model_name} best score: {best_score}, num_features: {len(best_features)},
best features: {best_features}')

Linear Regression best score: 12.526764152572579, num_features: 5, best features: ['V7', 'V17', 'V10', 'V4', 'V25']
Ridge best score: 12.526763926952748, num_features: 5, best features: ['V7', 'V17', 'V10', 'V4', 'V25']
Lasso best score: 12.535082495159886, num_features: 4, best features: ['V7', 'V17', 'V10', 'V4']
ElasticNet best score: 12.52996843930851, num_features: 4, best features: ['V7', 'V17', 'V10', 'V4']
SVR best score: 12.536671680166233, num_features: 16, best features: ['V7', 'V17', 'V10', 'V4', 'V25', 'V8', 'V23', 'V21', 'V3', 'V5', 'V13', 'V11', 'V22', 'V16', 'V19', 'V6']
Gradient Boosting best score: 12.62843426586662, num_features: 1, best features: ['V7']

```

Filter Method를 사용하여 각 모델 별로 최적의 피처를 선정하였다.

Forward Selection

```

import numpy as np
import pandas as pd
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.svm import SVR

```

```

from sklearn.feature_selection import SequentialFeatureSelector
from sklearn.metrics import make_scorer, mean_absolute_error

# 데이터 분할
X = train.drop('Target', axis=1) # Target 컬럼 제외
y = train['Target'] # Target 컬럼

# 모델 정의
models = {
    'Linear Regression': LinearRegression(),
    'Ridge': Ridge(random_state=42),
    'Lasso': Lasso(random_state=42),
    'ElasticNet': ElasticNet(random_state=42),
    'SVR': SVR(),
    'Gradient Boosting': GradientBoostingRegressor(random_state=42)
}

# MAE를 평가 기준으로 사용하기 위해 scorer 정의
mae_scorer = make_scorer(mean_absolute_error)

# 각 모델에 대해 최적의 특성 선택
for model_name, model in models.items():

    # 최적의 특성을 찾기 위한 변수 초기화
    best_score = float('inf')
    best_features = []

    # 최적의 특성 선택
    sfs = SequentialFeatureSelector(model, n_features_to_select=5, direction='forward')
    fit = sfs.fit(X, y)

    # 선택된 피처
    fs = X.columns[fit.support_].tolist()

    # 선택된 feature 데이터프레임 생성
    X_selected = X.iloc[:, fit.get_support()]

    # 교차 검증
    mae = cross_val_score(model, X_selected, y, cv=5, scoring=mae_scorer).mean()

    # 결과 출력
    print(f'{model_name} best score: {mae}, best features: {fs}')

Linear Regression best score: 12.524648835270032, best features: ['V3', 'V10', 'V17', 'V25', 'V26']
Ridge best score: 12.524648903536, best features: ['V3', 'V10', 'V17', 'V25', 'V26']
Lasso best score: 12.534578335004287, best features: ['V1', 'V3', 'V5', 'V10', 'V17']
ElasticNet best score: 12.528527107451348, best features: ['V1', 'V8', 'V10', 'V17', 'V25']
SVR best score: 12.540622387546296, best features: ['V3', 'V4', 'V16', 'V23', 'V25']
Gradient Boosting best score: 12.58342619636261, best features: ['V2', 'V8', 'V11', 'V16', 'V21']

```

다음으로 SequentialFeatureSelector()를 사용하여 Forward Selection을 진행하고 피처의 개수를 5, 10, 15개로 늘려 진행하였다.

Backward Elimination

```
import pandas as pd
import numpy as np
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.feature_selection import RFE
from sklearn.metrics import make_scorer, mean_absolute_error

# 데이터 분할
X = train.drop('Target', axis=1) # Target 컬럼 제외
y = train['Target'] # Target 컬럼

# 모델 정의
models = {
    'Linear Regression': LinearRegression(),
    'Ridge': Ridge(random_state=42),
    'Lasso': Lasso(random_state=42),
    'ElasticNet': ElasticNet(random_state=42),
    'Gradient Boosting': GradientBoostingRegressor(random_state=42)
}

# MAE를 평가 기준으로 사용하기 위해 scorer 정의
mae_scorer = make_scorer(mean_absolute_error)

# RFE를 사용하여 feature selection
for model_name, model in models.items():
    rfe = RFE(model, n_features_to_select=5)
    fit = rfe.fit(X, y)

    fs = X.columns[fit.support_].tolist()
    X_selected = X.iloc[:, fit.get_support()]

    # 선택된 feature로 cross-validation 수행
    score = cross_val_score(model, X_selected, y, cv=5, scoring=mae_scorer)

    print(f'{model_name} score: {score.mean()}, selected features: {fs}')
Linear Regression score: 12.543496571231898, selected features: ['V5', 'V14', 'V18', 'V22', 'V24']
Ridge score: 12.543495934779815, selected features: ['V5', 'V14', 'V18', 'V22', 'V24']
Lasso score: 12.53489854284295, selected features: ['V2', 'V4', 'V7', 'V10', 'V17']
ElasticNet score: 12.530659655893988, selected features: ['V4', 'V7', 'V10', 'V17', 'V25']
Gradient Boosting score: 12.780808193079931, selected features: ['V1', 'V6', 'V16', 'V24', 'V25']
```

마지막으로 RFE()를 사용하여 Backward Elimination 방식으로 피처를 선택하였다. SVR 모델의 경우 RFE를 사용할 수 없기 때문에 제외하고 진행하였다. Backward Elimination도 피처의 개수를 5, 10, 15로 늘려 확인해보았다.

결과 비교

Basic Data

	base	filter	for-5	for-10	for-15	back-5	back-10	back-15
Linear	12.6260	12.5267	12.5246	12.5287	12.5386	12.5434	12.5332	12.5620
Ridge	12.6258	12.5267	12.5246	12.5287	12.5386	12.5434	12.5332	12.5620
Lasso	12.5802	12.5350	12.5345	12.5345	12.5345	12.5348	12.5450	12.5539
ElasticNet	12.5829	12.5299	12.5285	12.5285	12.5285	12.5306	12.5388	12.5477
SVR	12.5641	12.5366	12.5406	12.5375	12.5350			
GB	12.6333	12.6284	12.5834	12.6099	12.5826	12.7808	12.5762	12.6074

Basic Time Data

	base	filter	for-5	for-10	for-15	back-5	back-10	back-15
Linear	12.6371	12.5267	12.5246	12.5297	12.5375	12.5434	12.5395	12.5416
Ridge	12.6370	12.5267	12.5246	12.5297	12.5375	12.5434	12.5395	12.5416
Lasso	12.5856	12.5350	12.5345	12.5345	12.5345	12.5348	12.5455	12.5461
ElasticNet	12.5938	12.5299	12.5285	12.5285	12.5285	12.5306	12.5389	12.5449
SVR	12.5799	12.5399	12.5347	12.5280	12.5299			
GB	12.6414	12.6114	12.6235	12.6009	12.5953	12.6786	12.6327	12.6780

One-hot + PCA Data

	base	filter	for-5	for-10	for-15	back-5	back-10	back-15
Linear	12.7023	12.4912	12.5071	12.4892	12.4867	12.5205	12.5102	12.5207
Ridge	12.6981	12.4906	12.5072	12.4894	12.4869	12.5209	12.5095	12.5116
Lasso	12.5802	12.5350	12.5345	12.5345	12.5345	12.5348	12.5428	12.5428
ElasticNet	12.5829	12.5299	12.5285	12.5285	12.5285	12.5306	12.5389	12.5389
SVR	12.5643	12.5287	12.5221	12.4746	12.4593			
GB	12.6956	12.5269	12.4980	12.4741	12.4719	12.7076	12.6287	12.6378

각 데이터별로 feature selection을 진행한 결과는 위와 같다. 결과를 보면 피처를 선택하여 일부만 학습한 경우 5-fold의 결과가 좋아진 것을 알 수 있다. 또한 피처 선택을 하지 않고

전체 피처를 학습한 결과를 보면 Base Data가 성능이 좋은데, 피처 선택을 진행한 경우 Onehot+PCA Data의 성능이 좋은 것을 알 수 있다. 결과적으로 Basic Data와 Basic Time Data는 Linear Regression과 Ridge 모델의 성능이 좋고, One-hot+PCA Data는 SVR이 성능이 좋게 나왔다.

3) 중간 제출 결과 비교

Model	Data	5-fold	submission
SVR	Basic Data	12.5641	12.66
	Basic Time Data	12.5799	12.64
	One-hot + PCA Data	12.5643	12.66
	One-hot + PCA + vif 제거	12.5340	12.70
	One-hot + PCA + forward 15	12.4593	12.71

테스트 결과 가장 성능이 좋았던 경우는 SVR 모델을 사용하여 Feature Selection을 진행한 경우였다. 앞에서 테스트한 결과를 바탕으로 성능이 높았던 SVR 모델을 제출해보았다. 그 결과 5-fold로 확인한 성능이 좋았던 모델이 제출하여 확인한 성능이 매우 떨어지는 것을 확인할 수 있었다. 5-fold에서는 Basic Time Data보다 Basic Data의 성능이 더 좋았는데, 제출 결과 반대로 나왔으며, Feature Selection은 진행한 경우 성능이 더 나빠지는 것을 확인할 수 있었다. 이를 통해 시계열 데이터를 포함하여 학습한 경우가 포함하지 않은 경우보다 성능이 좋다는 것을 확인할 수 있었고, SVR 모델은 이상치의 영향을 많이 받았거나 과적합이 발생하여 문제가 발생했다는 것을 알 수 있었다.

Model	Data	5-fold	submission
Linear Regression	Basic Time Data	12.6371	12.62
Lasso	Basic Time Data	12.5856	12.60
	One-hot + PCA Data	12.5802	12.60
	One-hot + PCA Data + vif 제거	12.5246	12.60
	Basic Time Data + forward 5	12.5345	12.62
ElasticNet	Basic Time Data	12.5938	12.62

오버피팅을 해결할 수 있는 모델은 L1과 L2 규제를 사용한 Lasso, Ridge, ElasticNet이다. 비교를 위해 규제를 사용하지 않은 Linear Regression을 제출한 뒤 Lasso, ElasticNet을

제출하여 비교해보았다. 그 결과 L1 규제를 사용한 Lasso의 성능이 가장 좋았다. L1과 L2 규제를 모두 사용한 ElasticNet보다 성능이 좋은 것으로 보다 L1 규제를 사용한 경우 오버피팅을 효과적으로 해결할 수 있는 것 같다. 그래서 Lasso를 응용한 모델들을 추가적으로 테스트해보았다.

4) 추가 테스트

Lasso

```
import pandas as pd
import numpy as np
from group_lasso import GroupLasso
from sklearn.linear_model import Lasso
from sklearn.metrics import make_scorer, mean_absolute_error
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import StandardScaler

# 피쳐와 타겟 설정
X = train.drop('Target', axis=1)
y = train['Target']

# 그룹 설정 (예시로 그룹을 3개로 나눈다고 가정)
n_groups = 3
groups = np.concatenate([np.full(X.shape[1] // n_groups, i) for i in range(n_groups)])
groups = np.concatenate([groups, np.full(X.shape[1] - len(groups), n_groups-1)]) # 나머지 그룹 처리

# 피쳐 스케일링
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Group Lasso 모델 학습 및 5-fold 교차 검증
group_lasso_model = GroupLasso(groups=groups, fit_intercept=True,
old_regularisation=True, tol=1e-4)

# MAE 평가를 위한 scorer 정의
mae_scorer = make_scorer(mean_absolute_error)

# Group Lasso 5-fold 교차 검증
group_lasso_scores = cross_val_score(group_lasso_model, X_scaled, y, cv=5,
scoring=mae_scorer)
print(f'Group Lasso Mean Absolute Error (5-fold CV): {-np.mean(group_lasso_scores)} ± {np.std(group_lasso_scores)}')
```

```

# Adaptive Lasso 모델 학습 및 5-fold 교차 검증
# 초기 Lasso 모델로 계수 추정
initial_model = Lasso(alpha=0.1, random_state=42)
initial_model.fit(X_scaled, y)

# 각 계수에 대한 가중치 계산 (1/|coeff| 형태) - 0인 경우 처리
weights = np.zeros_like(initial_model.coef_) # 초기화
for i, coef in enumerate(initial_model.coef_):
    if coef != 0:
        weights[i] = 1 / np.abs(coef) # 0이 아닐 경우
    else:
        weights[i] = 0.01 # 0일 경우 0.01로 설정

# Adaptive Lasso 모델 학습 (가중치 적용)
adaptive_lasso_model = Lasso(alpha=0.1, random_state=42)

# Adaptive Lasso 5-fold 교차 검증 (가중치를 적용하여 피처를 조정)
adaptive_lasso_scores = cross_val_score(adaptive_lasso_model, X_scaled * weights, y,
cv=5, scoring=mae_scorer)
print(f'Adaptive Lasso Mean Absolute Error (5-fold CV): {-
np.mean(adaptive_lasso_scores)} ± {np.std(adaptive_lasso_scores)}')

Group Lasso Mean Absolute Error (5-fold CV): -12.541458574029678 ± 0.16327871226084678
Adaptive Lasso Mean Absolute Error (5-fold CV): -12.539553750625508 ± 0.16010837574502487

```

첫 번째로 Adaptive Lasso와 Group Lasso를 테스트하여 성능이 더 좋은 Adaptive Lasso를 제출해보았다. 그 결과 Lasso의 성능이 더 좋은 것을 확인하였다.

하이퍼파라미터 튜닝

```

import pandas as pd
import numpy as np
from sklearn.linear_model import Lasso
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
import optuna

# Optuna objective function
def objective(trial):
    # alpha 값을 Optuna에서 추천
    alpha = trial.suggest_float('alpha', 1e-5, 1e2, log=True) # 로그 스케일로 alpha 샘플링
    random_state = trial.suggest_int('random_state', 1, 10000)
    model = Lasso(alpha=alpha, random_state=random_state, max_iter=10000) # max_iter
증가

```

```

# 5-fold cross-validation을 통한 성능 평가
scores = cross_val_score(model, X_train_scaled, y_train, cv=5,
scoring='neg_mean_absolute_error')
mean_score = -scores.mean() # 평균 MAE

return mean_score

# Optuna study 생성
study = optuna.create_study(direction='minimize')
study.optimize(objective, n_trials=1000) # 100번의 실험 수행

# 최적의 파라미터와 점수 출력
print("Best parameters: ", study.best_params)
print("Best score: ", study.best_value)

# 최적의 파라미터로 모델 학습
best_model = Lasso(alpha=study.best_params['alpha'],
random_state=study.best_params['random_state'], max_iter=10000)
best_model.fit(X_train, y_train)

# 훈련 데이터와 검증 데이터에 대한 예측
train_pred = best_model.predict(X_train)
val_pred = best_model.predict(X_val)
test_pred = best_model.predict(X_test)

# 5-fold 교차 검증으로 훈련 데이터 평가
cv_scores = cross_val_score(best_model, X_train_scaled, y_train, cv=5,
scoring='neg_mean_absolute_error')
train_score = -cv_scores.mean() # 평균 MAE

val_score = mean_absolute_error(y_val, val_pred)
print(f'Train Score (5-Fold CV): {train_score}')
print(f'Validation Score: {val_score}')

```

앞에서 Lasso의 성능이 더 좋은 것을 확인했기 때문에 Optuna를 사용하여 Lasso의 하이퍼파라미터 튜닝을 진행하여 성능을 높이려고 시도해보았다. 하지만 Lasso의 하이퍼파라미터가 alpha밖에 없기 때문에 이 방법으로 성능을 높이는 것이 어려웠다.

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import Lasso, Ridge
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import train_test_split, cross_val_score

```

```

from sklearn.preprocessing import StandardScaler

# 피처와 타겟 설정
X = train.drop('Target', axis=1)
y = train['Target']

# 훈련 세트와 검증 세트 분리
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)

# 피처 스케일링
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)

# alpha 값을 수동으로 정의 (예: 0.01, 0.1, 1.0 등)
alpha_space = np.arange(0.01, 1.01, 0.01)

# Lasso 모델에 대한 교차 검증 수행
lasso_scores = []
lasso = Lasso() # normalize=True 제거
for alpha in alpha_space:
    lasso.alpha = alpha
    # 10-fold cross-validation을 통한 성능 평가
    val = np.mean(cross_val_score(lasso, X_train_scaled, y_train, cv=10,
scoring='neg_mean_absolute_error'))
    lasso_scores.append(-val) # MAE는 음수로 반환되므로 부호를 변경

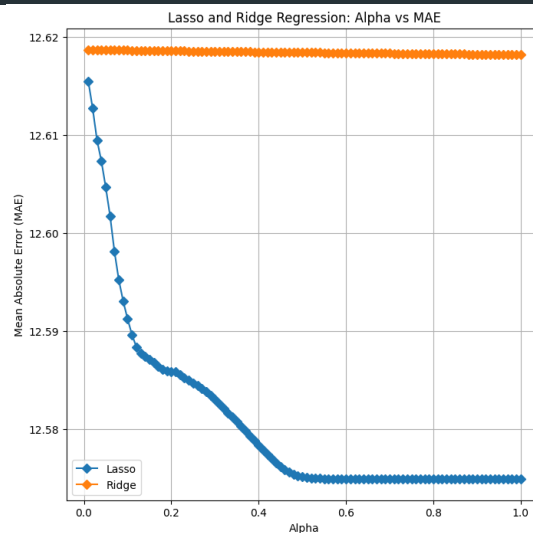
# Ridge 모델에 대한 교차 검증 수행
ridge_scores = []
ridge = Ridge() # normalize=True 제거
for alpha in alpha_space:
    ridge.alpha = alpha
    val = np.mean(cross_val_score(ridge, X_train_scaled, y_train, cv=10,
scoring='neg_mean_absolute_error'))
    ridge_scores.append(-val) # MAE는 음수로 반환되므로 부호를 변경

# 시각화
plt.figure(figsize=(8, 8))
plt.plot(alpha_space, lasso_scores, marker='D', label="Lasso")
plt.plot(alpha_space, ridge_scores, marker='D', label="Ridge")
plt.xlabel('Alpha')
plt.ylabel('Mean Absolute Error (MAE)')
plt.title('Lasso and Ridge Regression: Alpha vs MAE')
plt.legend()
plt.grid()
plt.show()

# 최적의 alpha 값 찾기

```

```
best_alpha_lasso = alpha_space[np.argmin(lasso_scores)]
best_alpha_ridge = alpha_space[np.argmin(ridge_scores)]
print(f'Best Alpha for Lasso: {best_alpha_lasso}')
print(f'Best Alpha for Ridge: {best_alpha_ridge}')
```



alpha의 값에 따른 MAE 추이를 확인하기 위해 그래프를 그려 보았다. 그 결과 alpha의 값이 약 0.5 이상이 되면 MAE가 일정 수준 이하로 떨어지지 않는 것을 확인하였다. 그래서 model stacking을 통해 성능을 높여보려고 했다.

Model Stacking

```
import pandas as pd
import numpy as np
from sklearn.linear_model import Lasso, ElasticNet
from sklearn.svm import SVR
from sklearn.ensemble import StackingRegressor
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
import optuna

# 피쳐와 타겟 설정
X = train.drop('Target', axis=1)
y = train['Target']

# 훈련 세트와 검증 세트 분리
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)

# 피쳐 스케일링
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

```

X_val_scaled = scaler.transform(X_val)

def objective(trial):
    # 하이퍼파라미터 정의
    lasso_alpha = trial.suggest_float('lasso_alpha', 1e-4, 1e2, log=True)
    elastic_net_alpha = trial.suggest_float('elastic_net_alpha', 1e-4, 1e2, log=True)
    elastic_net_l1_ratio = trial.suggest_float('elastic_net_l1_ratio', 0, 1)
    svr_c = trial.suggest_float('svr_c', 1e-4, 1e2, log=True)
    random_state = trial.suggest_int('random_state', 0, 100)

    # 기본 모델 생성
    base_models = [
        ('lasso', Lasso(alpha=lasso_alpha, random_state=random_state,
max_iter=10000)), # max_iter 증가
        ('elastic_net', ElasticNet(alpha=elastic_net_alpha, l1_ratio=elastic_net_l1_ratio,
random_state=random_state, max_iter=10000)),
        ('svr', SVR(C=svr_c))
    ]

    # 스택킹 리그레서 생성
    stacking_model = StackingRegressor(estimators=base_models,
final_estimator=Lasso(random_state=random_state, max_iter=10000))

    # 모델 학습
    stacking_model.fit(X_train_scaled, y_train)

    # 교차 검증
    cv_scores = cross_val_score(stacking_model, X_train_scaled, y_train, cv=5,
scoring='neg_mean_absolute_error')
    return -cv_scores.mean()

# Optuna를 사용한 하이퍼파라미터 최적화
study = optuna.create_study(direction='minimize')
study.optimize(objective, n_trials=100)

# 최적의 하이퍼파라미터 출력
print("Best hyperparameters:", study.best_params)
print("Best score:", study.best_value)

# 최적의 하이퍼파라미터로 모델 재학습
best_params = study.best_params
model = StackingRegressor(
    estimators=[
        ('lasso', Lasso(alpha=best_params['lasso_alpha'],
random_state=best_params['random_state'])),
        ('elastic_net', ElasticNet(alpha=best_params['elastic_net_alpha'],
l1_ratio=best_params['elastic_net_l1_ratio'], random_state=best_params['random_state'])),
        ('svr', SVR(C=best_params['svr_c']))
    ]
)

```

```

    ],
    final_estimator=Lasso(random_state=best_params['random_state'])
)

# 모델 학습
model.fit(X_train_scaled, y_train)

# 훈련 데이터와 검증 데이터에 대한 예측
train_pred = model.predict(X_train_scaled)
val_pred = model.predict(X_val_scaled)

# 검증 데이터에 대한 MAE 계산
val_score = mean_absolute_error(y_val, val_pred)

print(f'Validation Score: {val_score}')

```

앞에서 성능이 괜찮았던 Lasso, ElasticNet, SVR을 사용하여 모델을 만들어보았다. 하이퍼파라미터 튜닝까지 하여 제출해보았으나 모델의 성능이 12.62로 Lasso 모델보다 성능이 좋지 못했다. 모델을 바꾸어 테스트해보았지만 유의미한 성과를 얻지 못했다.

Voting

```

import optuna
from sklearn.ensemble import VotingRegressor, RandomForestRegressor
from sklearn.linear_model import Lasso, ElasticNet
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import make_scorer, mean_absolute_error
from optuna.samplers import TPESampler

# 데이터 분할
X = train.drop('Target', axis=1) # Target을 제외한 모든 컬럼을 X로 지정
y = train['Target']             # Target 컬럼을 y로 지정

# train, test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# MAE를 평가 기준으로 사용하기 위해 scorer 정의
mae_scorer = make_scorer(mean_absolute_error)

# Optuna 목적 함수 정의
def objective(trial):
    # 하이퍼파라미터 선택
    lasso_alpha = trial.suggest_float('lasso_alpha', 0.001, 1.0)
    elastic_alpha = trial.suggest_float('elastic_alpha', 0.001, 1.0)
    elastic_l1_ratio = trial.suggest_float('elastic_l1_ratio', 0.1, 0.9)

```



```

rf_n_estimators = trial.suggest_int('rf_n_estimators', 50, 300)
rf_max_depth = trial.suggest_int('rf_max_depth', 3, 20)
rf_min_samples_split = trial.suggest_int('rf_min_samples_split', 2, 10)
rf_min_samples_leaf = trial.suggest_int('rf_min_samples_leaf', 1, 10)
rf_max_features = trial.suggest_categorical('rf_max_features', ['sqrt', 'log2', None])
rf_random_state = trial.suggest_int('rf_random_state', 1, 100)

# 모델 정의
lasso = Lasso(alpha=lasso_alpha, random_state=rf_random_state)
elastic = ElasticNet(alpha=elastic_alpha, l1_ratio=elastic_l1_ratio,
random_state=rf_random_state)
rf = RandomForestRegressor(
    n_estimators=rf_n_estimators,
    max_depth=rf_max_depth,
    min_samples_split=rf_min_samples_split,
    min_samples_leaf=rf_min_samples_leaf,
    max_features=rf_max_features,
    random_state=rf_random_state
)

# VotingRegressor 정의
voting = VotingRegressor([('lasso', lasso), ('elastic', elastic), ('rf', rf)])

# 5-fold 교차 검증으로 평가
scores = cross_val_score(voting, X_train, y_train, cv=5, scoring=mae_scorer)
return scores.mean()

# Optuna 최적화 실행
study = optuna.create_study(direction='minimize', sampler=TPESampler())
study.optimize(objective, n_trials=50)

# 최적 하이퍼파라미터 출력
print("Best parameters:", study.best_params)

# 최적 하이퍼파라미터로 VotingRegressor 학습
best_lasso = Lasso(alpha=study.best_params['lasso_alpha'],
random_state=study.best_params['rf_random_state'])
best_elastic = ElasticNet(alpha=study.best_params['elastic_alpha'],
l1_ratio=study.best_params['elastic_l1_ratio'],
random_state=study.best_params['rf_random_state'])
best_rf = RandomForestRegressor(
    n_estimators=study.best_params['rf_n_estimators'],
    max_depth=study.best_params['rf_max_depth'],
    min_samples_split=study.best_params['rf_min_samples_split'],
    min_samples_leaf=study.best_params['rf_min_samples_leaf'],
    max_features=study.best_params['rf_max_features'],
    random_state=study.best_params['rf_random_state'])

```

```

)

voting_best = VotingRegressor([('lasso', best_lasso), ('elastic', best_elastic), ('rf',
best_rf)])

# 최적화된 모델 학습 및 평가
voting_best.fit(X_train, y_train)
y_pred = voting_best.predict(X_test)

train_score = mean_absolute_error(y_train, voting_best.predict(X_train))
test_score = mean_absolute_error(y_test, y_pred)

print(f'Optimized Voting Regressor - Train MAE: {train_score}, Test MAE: {test_score}')

```

Model Stacking에서 좋은 성능을 보이는 모델을 찾지 못해 VotingRegressor를 사용하여 새로운 모델을 만들고 Optuna를 사용하여 하이퍼파라미터 튜닝을 진행해보았다. 하이퍼파라미터 튜닝을 통해 성능을 높일 수 있었지만 제출 결과 12.61로 Lasso보다 성능이 떨어지는 것을 확인할 수 있었다. Lasso 모델은 Feature Selectino이 모델의 성능에 영향을 미치지 않지만 이번 모델은 Voting을 사용하여 다른 모델들도 사용하기 때문에 Feature Selection이 도움이 될 것으로 예상하였다. 그래서 앞에서 사용한 FowardSelection 코드로 5, 10, 15, 20개의 성능이 가장 좋은 피쳐 조합을 찾아보았다. 그 결과 Lasso 보다 성능이 좋은 모델을 찾을 수 있었다.

5) 추가 테스트 제출 결과 비교

Model	Data	5-fold	submission
Adaptive Lasso	Basic Time Data	12.5395	12.62
Model Stacking (SVR, ElasticNet, Lasso)	Basic Time Data	12.5713	12.62
Voting (Lasso, ElasticNet, RandomForest)	Basic Time Data	12.5398	12.61
	Basic Time Data + for5	12.5306	12.60
	Basic Time Data + for10	12.5281	12.59
	Basic Time Data + for15	12.5326	12.60
	Basic Time Data + for20	12.5287	12.60

추가로 테스트한 모델들을 제출하여 얻은 결과들이다. 제출 결과 voting 모델을 Forward Selection으로 5개의 피처를 추출한 결과가 12.60으로 Lasso보다 아주 조금 더 성능이 좋았다. 그리고 Feature Selection으로 10개의 피처를 추출한 결과 12.59로 가장 좋은 성능이 나왔다. 그리고 피처의 개수가 증가하면 성능이 나빠졌다.

4. DL Modeling

머신러닝 모델에서 성능이 좋았던 Lasso는 모델의 성능을 올리는 데에 한계가 있다. Lasso는 L1 규제를 사용하여 과적합을 방지하기 때문에 성능이 좋은 것으로 추측된다. 따라서 딥러닝 모델에 배치 정규화, 드랍 아웃, L1 정규화를 사용하여 과적합을 방지하는 모델을 만들어 테스트해보았다.

1) Sequential

모델

```
import numpy as np
import pandas as pd
from sklearn.model_selection import KFold
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization, Input
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.regularizers import l1 # L1 정규화 임포트

# 랜덤 시드 고정
np.random.seed(42) # NumPy 랜덤 시드
tf.random.set_seed(42) # TensorFlow 랜덤 시드

# 데이터 로드
train = pd.read_csv('./train.csv')

# yymm 컬럼을 날짜 형식으로 변환 (연도는 임의로 설정)
train['yyymm'] = pd.to_datetime('2024' + train['yyymm'], format='%Y%m%d %H:%M')

# day, hour, minute 컬럼 생성
train['day'] = train['yyymm'].dt.day # 일
train['hour'] = train['yyymm'].dt.hour # 시
train['minute'] = train['yyymm'].dt.minute # 분

# weekday 컬럼 생성
train['weekday'] = train['day'] % 7 # 요일 (0: 월요일, 1: 화요일, ..., 6: 일요일)

# yymm 컬럼 삭제
train.drop('yyymm', axis=1, inplace=True)

# 데이터 분할
```

```

X = train.drop('Target', axis=1)    # Target을 제외한 모든 컬럼을 X로 지정
y = train['Target']                # Target 컬럼을 y로 지정

# 특징 스케일링
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# K-fold cross-validation 설정 (5-fold)
kf = KFold(n_splits=5, shuffle=True, random_state=42)

# 설정한 하이퍼파라미터
learning_rates = [0.001, 0.01]    # 여러 learning rate
batch_sizes = [16, 32, 64]        # 여러 batch size

# 성능 비교를 위한 결과 저장
results = []

# 딥러닝 회귀 모델 정의 함수 (DNN)
def create_model(learning_rate):
    model = Sequential([
        Input(shape=(X_scaled.shape[1],)), # Input layer로 input shape 지정
        Dense(64, activation='relu', kernel_regularizer=l1(0.01)), # DNN 레이어 (크기 축소)
        BatchNormalization(),
        Dropout(0.3),
        Dense(32, activation='relu', kernel_regularizer=l1(0.01)), # DNN 레이어 (크기 축소)
        BatchNormalization(),
        Dropout(0.3),
        Dense(1) # 회귀 모델이므로 활성화 함수 없이 출력
    ])
    model.compile(optimizer=Adam(learning_rate=learning_rate),
loss='mean_absolute_error')
    return model

# 다양한 learning rate와 batch size에 대해 성능 비교
for learning_rate in learning_rates:
    for batch_size in batch_sizes:
        mae_scores = []

        # 5-fold 교차 검증
        for train_index, val_index in kf.split(X_scaled):
            X_train, X_val = X_scaled[train_index], X_scaled[val_index]
            y_train, y_val = y[train_index], y[val_index]

            # 모델 생성 및 학습
            model = create_model(learning_rate)

            # EarlyStopping 설정

```

```

early_stopping = EarlyStopping(monitor='val_loss', patience=10,
restore_best_weights=True)

model.fit(X_train, y_train, epochs=100, batch_size=batch_size,
validation_data=(X_val, y_val), verbose=0, callbacks=[early_stopping])

# 예측 및 MAE 계산
y_pred = model.predict(X_val)
mae = mean_absolute_error(y_val, y_pred)
mae_scores.append(mae)

# 평균 MAE 결과 저장
mean_mae = np.mean(mae_scores)
results.append((learning_rate, batch_size, mean_mae))
print(f'Learning Rate: {learning_rate}, Batch Size: {batch_size}, 5-fold MAE:
{mean_mae:.4f}')

# 결과 출력
print("\nResults:")
for lr, bs, mae in results:
    print(f'Learning Rate: {lr}, Batch Size: {bs}, Mean MAE: {mae:.4f}')

```

```

Results:
Learning Rate: 0.001, Batch Size: 16, Mean MAE: 12.5547
Learning Rate: 0.001, Batch Size: 32, Mean MAE: 12.5331
Learning Rate: 0.001, Batch Size: 64, Mean MAE: 12.5956
Learning Rate: 0.01, Batch Size: 16, Mean MAE: 12.5428
Learning Rate: 0.01, Batch Size: 32, Mean MAE: 12.5340
Learning Rate: 0.01, Batch Size: 64, Mean MAE: 12.5405

```

tensorflow의 Sequential 모델을 사용했으며, 과적합을 방지하기 위해 배치 정규화, 드롭아웃을 사용하였다. 또한 Lasso가 성능이 좋았기 때문에 L1 정규화를 사용하였다. 또한 성능 향상을 위해 Early Stopping을 사용하여 성능을 높였다. 추가적으로 Sequential 모델의 층 수를 늘리고 VIF가 큰 컬럼을 삭제하여 테스트를 진행해보았다.

결과 비교

Basic Model			Complex Model			VIF Deletion		
Learning Rate	Batch Size	MAE	Learning Rate	Batch Size	MAE	Learning Rate	Batch Size	MAE
0.001	16	12.5547	0.001	16	12.5347	0.001	16	12.5226
0.001	32	12.5331	0.001	32	12.5554	0.001	32	12.5216

0.001	64	12.5956	0.001	64	12.5372	0.001	64	12.5062
0.01	16	12.5428	0.01	16	12.5416	0.01	16	12.5503
0.01	32	12.5340	0.01	32	12.5363	0.01	32	12.5409
0.01	64	12.5405	0.01	64	12.5450	0.01	64	12.5465

Learning Rate, Batch Size를 변경하며 성능을 평가해보았다. 그 결과 Batch Size가 작고 Learning Rate가 클수록 성능이 좋은 것을 확인하였다.

2) 제출 결과

Model	Learning Rate	Batch Size	submission
Basic Model	0.01	32	12.61
	0.001	32	12.76
Complex Model	0.01	64	12.61
	0.01	32	12.61
VIF Deletion	0.001	32	12.67

테스트한 모델들을 제출하여 성능을 비교해보았다. 그 결과 머신러닝 모델과 마찬가지로 테스트할 때 좋은 성능을 보인 모델이 제출했을 때 성능이 잘 나오지 않는 것을 확인할 수 있었다. 그리고 Lasso 모델이 12.60이었기 때문에 더 성능이 좋은 모델을 만들지 못했다.

5. 최종 결과

1) 제출 결과

```
from sklearn.ensemble import VotingRegressor
from sklearn.linear_model import Lasso, ElasticNet
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import make_scorer, mean_absolute_error
import pandas as pd

# 데이터 로드
train = pd.read_csv('./train.csv')
test = pd.read_csv('./test_set.csv')

# 연도를 1900으로 설정하여 yymm 컬럼을 날짜 형식으로 변환
train['yymm'] = pd.to_datetime('2024' + train['yymm'], format='%Y%m%d %H:%M')
test['yymm'] = pd.to_datetime('2024' + test['yymm'], format='%Y%m%d %H:%M')

# day, hour, minute 컬럼 생성
train['day'] = train['yymm'].dt.day # 일
train['hour'] = train['yymm'].dt.hour # 시
train['minute'] = train['yymm'].dt.minute # 분

test['day'] = test['yymm'].dt.day # 일
test['hour'] = test['yymm'].dt.hour # 시
test['minute'] = test['yymm'].dt.minute # 분

# weekday 컬럼 생성 (요일 계산)
train['weekday'] = train['day'] % 7 # 1은 화요일, 2는 수요일, ... 6은 일요일
test['weekday'] = test['day'] % 7 # 1은 화요일, 2는 수요일, ... 6은 일요일

# yymm 컬럼 삭제
train.drop('yymm', axis=1, inplace=True)
test.drop('yymm', axis=1, inplace=True)

X_train = train[['V2', 'V3', 'V7', 'V17', 'V20', 'V21', 'V23', 'V24', 'day', 'weekday']]
y_train = train['Target']

X_test = test[['V2', 'V3', 'V7', 'V17', 'V20', 'V21', 'V23', 'V24', 'day', 'weekday']]

# 모델 생성
lasso = Lasso(alpha=0.6779122520600722, random_state=30)
elastic = ElasticNet(alpha=0.9971090599987921, l1_ratio=0.44498452438450353,
random_state=30)
```



```

rf = RandomForestRegressor(
    n_estimators=168,
    max_depth=6,
    min_samples_split=2,
    min_samples_leaf=1,
    max_features=None,
    random_state=30
)

model = VotingRegressor([
    ('lasso', lasso),
    ('elastic', elastic),
    ('rf', rf)
])

model.fit(X_train, y_train)

train_pred = model.predict(X_train)
test_pred = model.predict(X_test)
submission = pd.DataFrame(test_pred, columns=['predict'])

train_score = mean_absolute_error(y_train, train_pred)
print(f'Train Score: {train_score}')
print(submission.head())

```

제출 결과는 test 데이터셋의 50%를 학습한 MAE 점수를 의미한다. 가장 성능이 좋은 모델의 코드는 위와 같다. yymm 컬럼의 시계열 데이터를 각각 다른 피처로 생성하고 요일을 나타내는 피처도 함께 생성한 데이터가 가장 성능이 좋았다. 그 중에서 Lasso 모델을 활용한 모델들의 성능이 좋았으며, VotingRegression을 사용하여 Lasso, ElasticNet, RandomForest의 평균 값을 사용한 모델의 성능이 가장 좋았다.

2) 경진대회 참여 소감

경진대회에 참여하여 성능을 높이는 과정에서 많은 어려움이 있었다. 처음으로 마주한 문제는 5-fold 교차 검증으로 테스트한 결과와 실제 제출했을 때 결과에 차이가 있었다는 점이다. 이 원인을 찾는 과정에서 SVR이 이상치에 민감하며, test 데이터셋의 분포가 train과 다르면 과적합이 발생할 수 있게 되었다. 그 과정에서 과적합을 해결할 수 있는 방법들을 찾아보게 되었다.

이번 경진대회 데이터에서는 L1규제를 사용하는 Lasso 모델의 성능이 좋았다. 그래서 딥러닝 모델을 사용하여 L1규제, 드랍아웃, 배치 정규화와 같은 과적합 해결방법들을 사용해

보았는데, 결국 기본적인 Lasso 모델보다 성능을 높이는데 실패했다. 또한 다양한 전처리 과정을 통해 피처를 늘리거나 줄여보았지만 성능이 좋아지지 않았다. 이 과정에서 만약 데이터에 대한 배경지식이 있어 피처들의 특성을 알고 있다면 전처리 과정에서 개선점을 더 찾을 수 있었을 것 같다는 생각을 하게 되었다.

모델을 개선하는 과정에서는 성능이 좋은 하나의 모델을 찾는 것도 중요하지만 다양한 모델들을 함께 사용하여 가장 성능이 좋은 모델을 찾아가는 과정이 중요하다는 것도 알게 되었다. 처음 Model Stacking을 시도한 뒤 제출 직전에 Voting을 테스트하게 되었는데, 해당 모델이 가장 성능이 좋았다. 만약 해당 모델을 먼저 시도하여 하이퍼파라미터 튜닝과 feature selection을 추가로 진행했다면 성능을 더 높일 수 있는 방법을 찾을 수 있었을 것 같다는 아쉬움이 있다. 하지만 이번 경험이 다른 경진대회에 참여할 때 많은 도움이 될 것 같다. 만약 다른 참가자들의 코드를 알게 된다면 분석하여 다른 분석 방법들을 더 배우고 싶다.